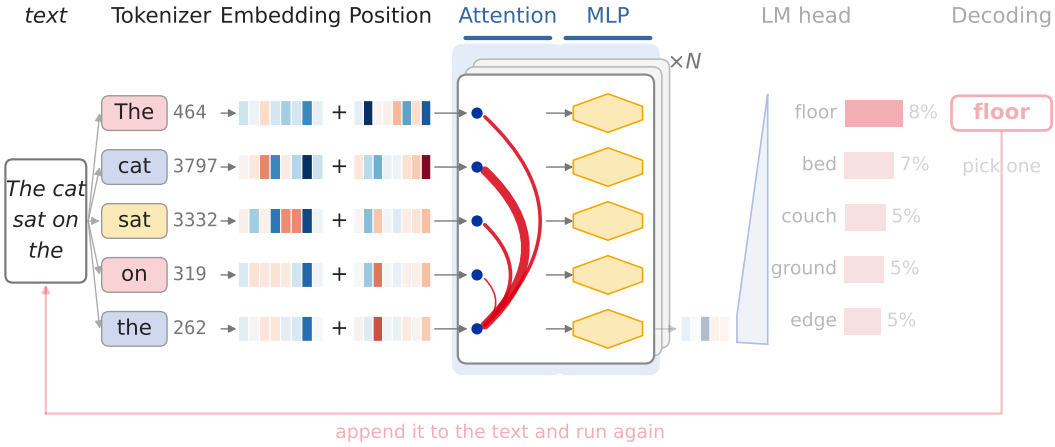
The Transformer block

Many heads, the MLP, and the residual stream

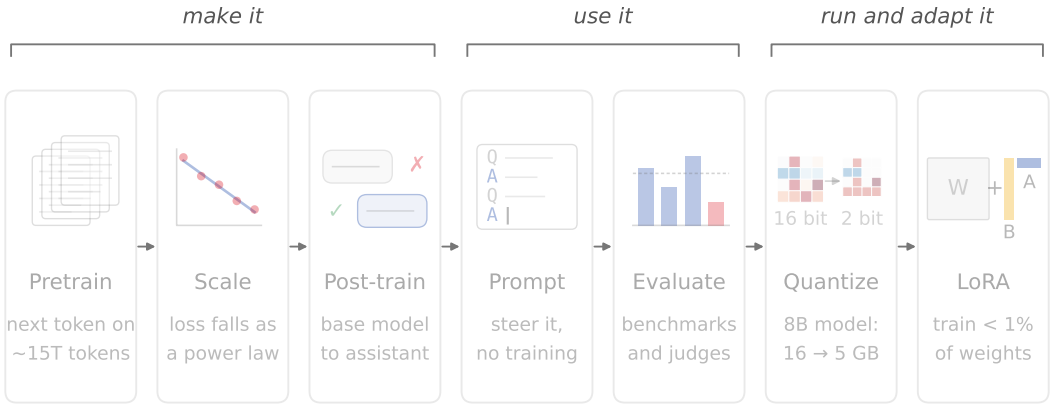
LLM chapter, session 4. Parts adapt 3Blue1Brown, “Deep Learning” chapters 6 and 7.

Where we are: one forward pass



Today: the rest of one layer - many heads and the MLP - and why it stacks 12 times.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

What does GPT-2 write next?

Four prompts for GPT-2 small, the model on our map. Predict its next word:

"Lionel Messi plays the sport of ..."

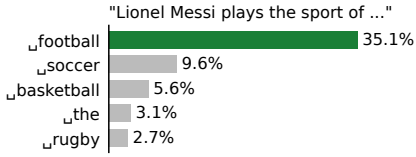
"Henrikh Mkhitaryan plays the sport of ..."

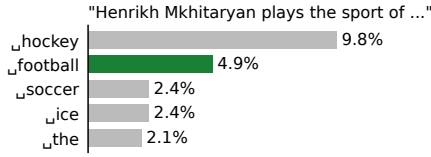
"Ella Fitzgerald was a famous American ..."

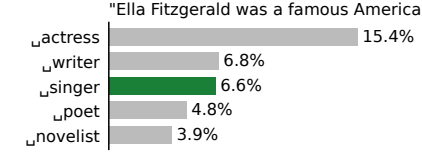
"Elliott Smith was a famous American ..."

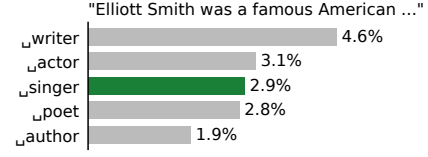
Predict: which of the four does it get right? And for the ones it gets wrong, what does it say instead?

Stored, half-stored, missing









Messi is stored: football, 35%. Mkhitaryan is missing: hockey first, football 5%. “Singer” is third for both musicians, but 6.6% against 2.9%. One run of the real GPT-2.

So where does a fact live?

No word in these sentences says “football” or “singer”.

Attention (LLM-3) only blends what the token vectors already hold. So the fact is either in a token’s own vector from the start (LLM-2’s table), or something adds it on the way through the layers.

And LLM-1 shows up again: “Lionel Messi” is 2 tokens, “Henrikh Mkhitaryan” is 7 pieces. The model has to put the name together before it can look anything up.

Today’s answer, as far as the evidence goes: mostly in the **MLP**, the half of each layer that LLM-3 never touched - and the bigger half.

Three things stand between one head and GPT-2

LLM-3 built **one head**: queries, keys, values, $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. To get from there to one layer of GPT-2:

1. **One head asks one question.** $\rightarrow$ many heads.
2. **Tokens talk, but never think alone.** $\rightarrow$ the MLP, where facts like Messi's sport seem to live.
3. **One layer is not a network.** $\rightarrow$ the residual stream and its norms; and a model that writes left to right must not read the future $\rightarrow$ the mask.

The three fixes bolted onto one head of attention *are* the Transformer block.

Outline

Many heads

The MLP: where facts live

The block

Many heads

One head asks one question.

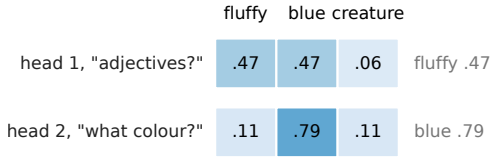
By hand: two heads, two questions

LLM-3's toy, slots [adjective, fluffy, blue, noun].

Head 1 asks "any adjectives?" (LLM-3's head): weights $0.47, 0.47, 0.06$; it hands over fluffiness, 0.47.

Head 2 asks "what colour?" (its key reads the blue slot): weights $0.11, 0.79, 0.11$; it hands over blueness, 0.79.

W_O puts both into the stream: creature gets $(0,\ 0.47,\ 0.79,\ 0)$.



W_O puts both numbers into the stream:
(adjective, fluffy, blue, noun) += (0, .47, .79, 0)

Two questions at once; W_O decides where each answer lands. Real heads hand over 64 numbers each, not one.

Run it h times, in parallel

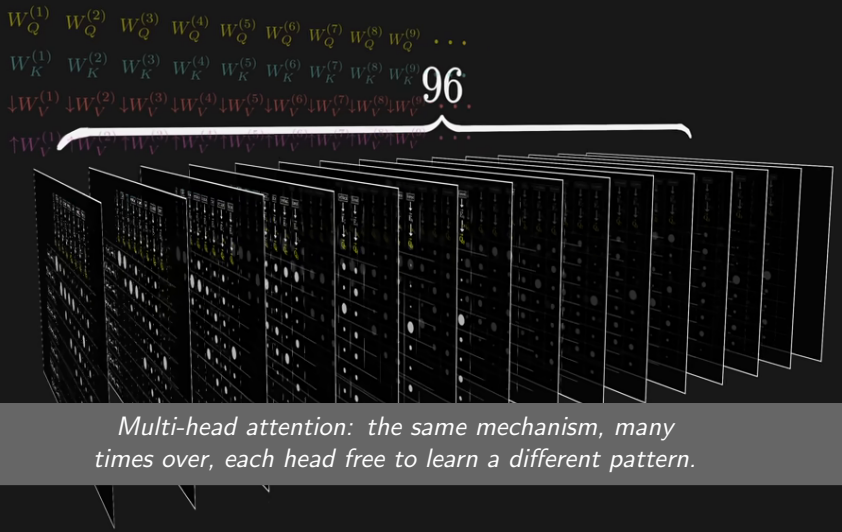
Nothing about one head changes. Learn h independent sets of W_Q, W_K, W_V, run them at the same time, put the outputs side by side, and mix them with one more matrix W_O:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O,$$
$$\text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

GPT-2: 12 heads $\times$ 64 numbers $=$ 768, the size of a token's vector (LLM-3's "why 64"). All twelve heads' W_Q, W_K, W_V sit in one $768 \times 2{,}304$ matrix; W_O is 768×768, so every output number can draw on every head. The same budget as one 768-wide head: you **split** it into 12 views, you do not add any.

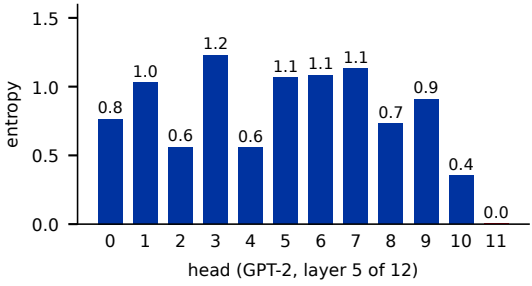
The heads do not talk to each other until the concatenation, so all twelve are one big batched matrix product.

Multi-headed attention



Multi-head attention: the same mechanism, many times over, each head free to learn a different pattern.

Do heads actually specialise, or is that a story?

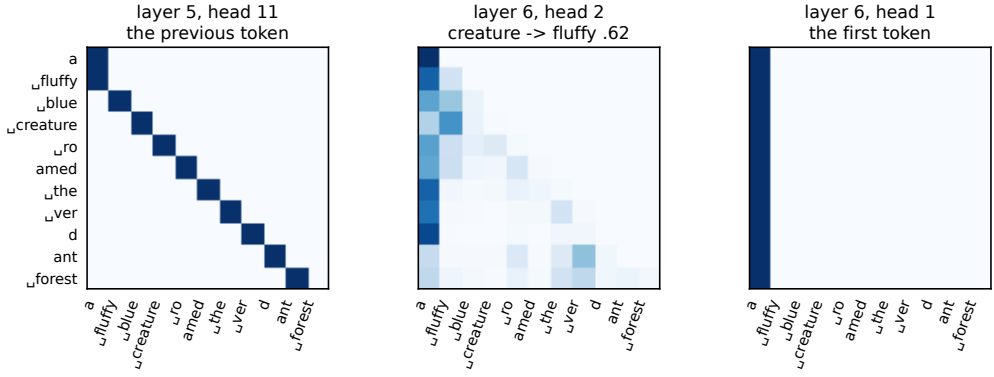


Measured: the real GPT-2, the creature sentence, the 12 heads of layer 5.

Attention entropy $= -\sum_j a_{ij} \log a_{ij}$, averaged over rows: low means a head looks at few tokens, high means it spreads out.

Head 11 scores **0.0**: every token puts all of its weight on a single token. Which one? Next slide.

Three real heads, one sentence

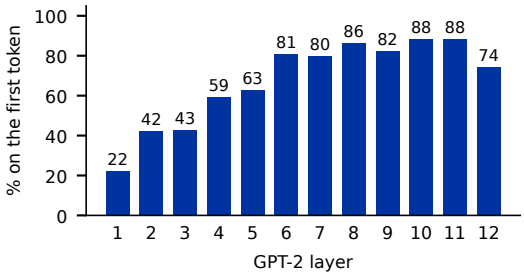


Left: every token looks exactly one token back (weight 1.00) - the head with entropy 0.

Middle: on this sentence, creature sends 62% of its attention to fluffy - LLM-3's toy, for real.

Right: everything goes to the first token, "a".

Attention sinks



The right-hand pattern is not rare: **39 of GPT-2's 144 heads** put over 90% of their attention on the first token of this sentence. The share grows with depth: 22% in layer 1, 88% in layer 10.

The usual explanation: softmax must hand out a total of 1, so a head with nothing useful to look at parks its weight on one token whose value changes little - the first.

This is called an **attention sink** (Xiao et al., ICLR 2024). Real heads are messier than the toy, exactly as LLM-3 warned.

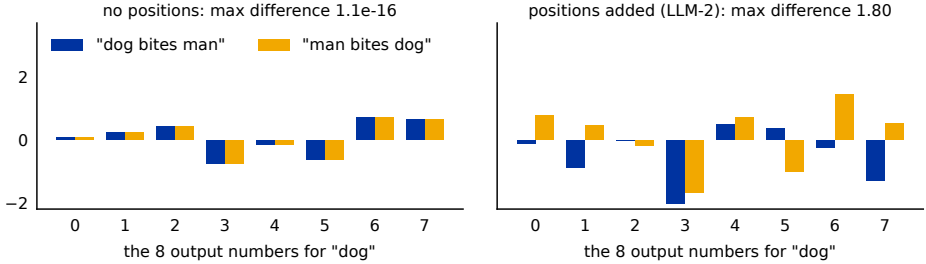
What the specialisations look like

Across models and layers, heads fall into recognisable jobs:

Kind of head	**What it attends to**
positional	the previous token, or the first one (both above)
syntactic	the verb belonging to this subject; the noun this adjective modifies
coreference	the name a pronoun refers to
lexical	the other half of a multi-token name (*Lionel* $\leftrightarrow$ *Messi*)

Read these labels with some suspicion. Nobody assigned these jobs; they are patterns *we* recognise afterwards, on the sentences we looked at, and plenty of heads do nothing so tidy. The measured claim is that the heads differ.

One head or twelve: still blind to order

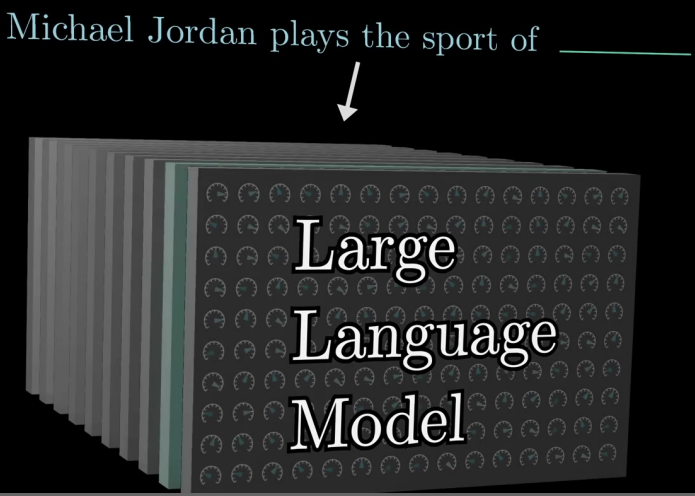


The proof LLM-2 promised. Same three word vectors, “dog bites man” vs “man bites dog”, one head: “dog” gets the same output in both, up to 1.1×10^{-16} - floating-point zero. Add positions (right) and the two differ.

Nowhere for order to enter: $\mathbf{q}_i \cdot \mathbf{k}_j$ uses only the two vectors, softmax works on a set, and the output is a sum. Permute the inputs and the outputs come back permuted, unchanged. That is why LLM-2 adds positions before the first layer.

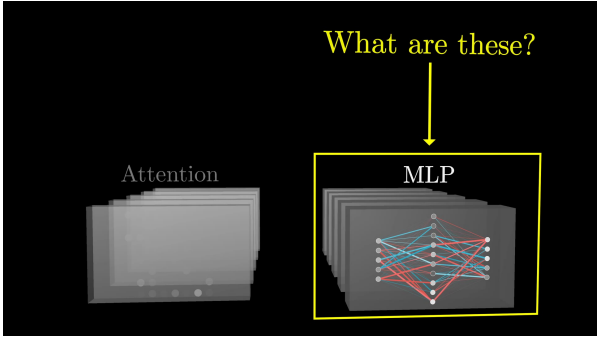
The MLP: where facts live

Attention moves information around. Something has to store it.



3Blue1Brown's version of our question: "Michael Jordan plays the sport of ___". Ours was Messi.

Where the MLP sits



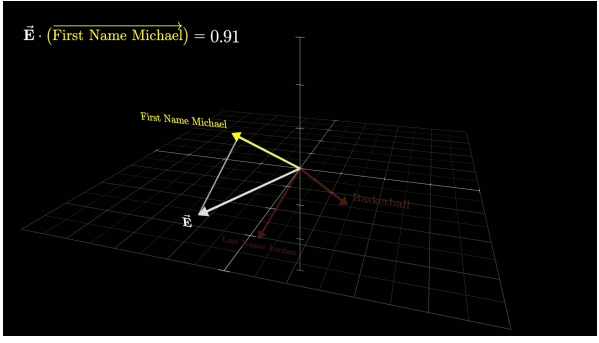
3Blue1Brown

After attention, every token goes through the **same** small network, **alone**: no token sees another inside the MLP.

It is the network from [41]: a linear layer, a nonlinearity, a linear layer.

Attention: tokens talk. MLP: each token thinks on its own.

A direction is a question

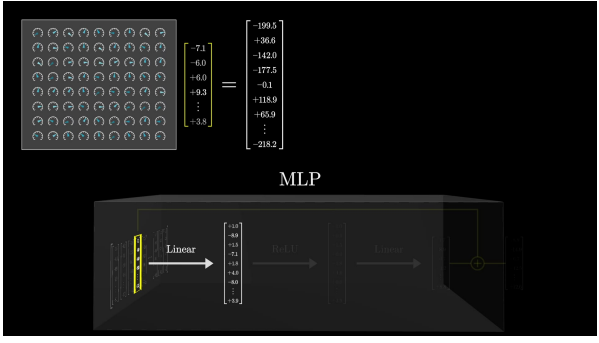


3Blue1Brown

LLM-2: directions in the vector space carry meaning. The dot product with a direction measures *how much* of that idea a vector holds.

Here: how much "first name Michael" is in this token's vector? 0.91: a lot.

Up-projection: each row asks a question

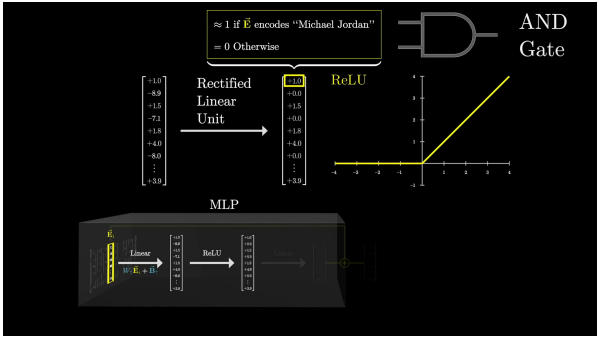


3Blue1Brown

The first layer computes $W_{\text{up}}\,\mathbf{x} + \mathbf{b}$. Each **row** of W_{up} is one such question, a dot product with one direction; the bias is a threshold.

GPT-2 asks **3,072** questions per token: four times its 768 numbers.

ReLU makes an AND gate



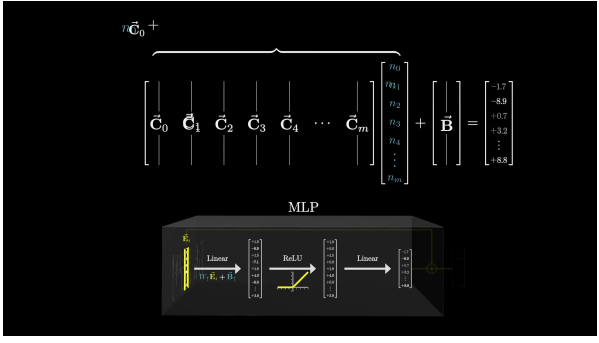
3Blue1Brown

Ask "first name Michael" + "last name Jordan", with bias -1: the sum is positive only if **both** are there.

ReLU sets everything negative to 0. What is left is a neuron that fires on "Michael AND Jordan" and stays silent otherwise.

GPT-2 actually uses GELU, a smoothed ReLU (slightly negative instead of exactly 0 below zero); the gate works the same way.

Down-projection: each column adds something



3Blue1Brown

The second layer maps the 3,072 neuron values back to 768 numbers. Each **column** of W_{down} is a direction; a neuron that fires adds its column into the stream.

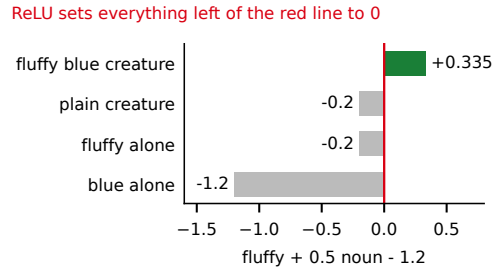
The "Michael Jordan" neuron's column can point toward "basketball". The fact is in the weights.

By hand: a neuron on creature

After LLM-3, creature is (0, 0.47, 0.47, 2.13). Give the toy one more slot, **cute**, that every word left at 0 - real vectors have 768 slots, most unnamed.

One neuron: $\mathrm{ReLU}(\text{fluffy} + 0.5\,\text{noun} - 1.2)$.

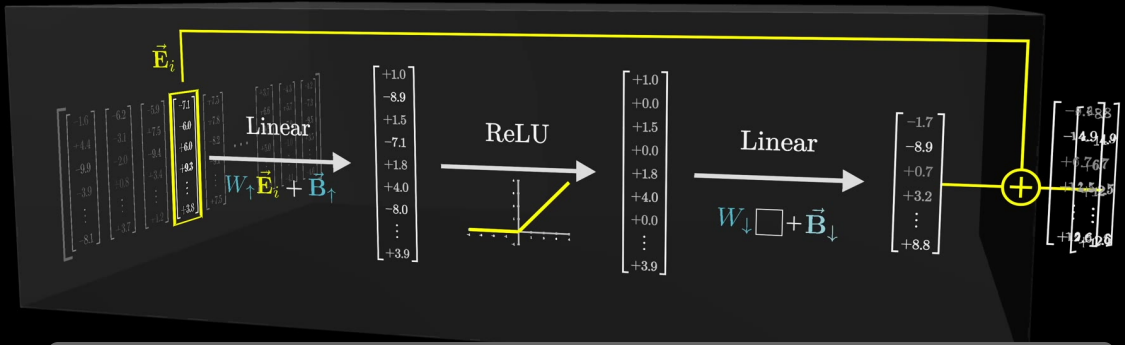
- this creature: $0.47 + 1.065 - 1.2 = \mathbf{0.335}$
- a plain creature $(0, 0, 0, 2)$: $-0.2 \to 0$
- fluffy alone: $-0.2 \to 0$; blue alone: $-1.2 \to 0$



Its column writes $3 \times 0.335 = 1.0$ into cute.

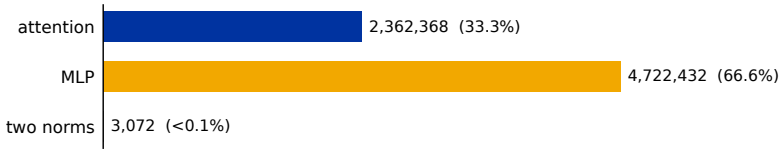
The neuron fires on "fluffy AND creature". That only works because attention first brought fluffy into creature's vector (LLM-3) - Michael and Jordan, the same way. Now creature carries a feature no word in the sentence had.

MLP



The whole MLP: linear, ReLU, linear, then add the result back into the stream. The same weights for every token.

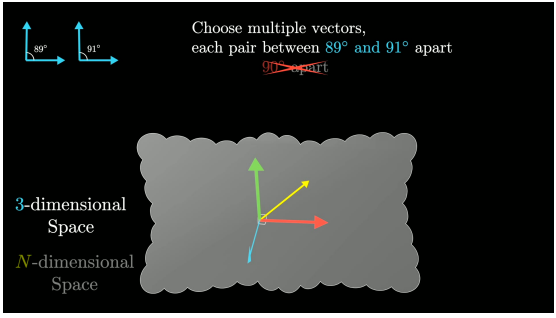
Two thirds of the weights



One GPT-2 block, counted from its weight file: the MLP holds **66.6%**. 3Blue1Brown counts the same share for GPT-3. Last lecture was about the smaller half.

Evidence that facts sit in these layers: the MLPs behave like key-value memories (Geva et al., EMNLP 2021), and editing one mid-layer MLP can change what GPT-2 XL says about a fact (Meng et al., NeurIPS 2022). And the cold open: models answer worse about people and things that are rare in their training text (Kandpal et al., ICML 2023).

More facts than dimensions?

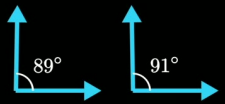


3Blue1Brown

768 numbers per token, yet a model knows far more than 768 things. How?

In many dimensions, directions do not have to be exactly perpendicular to stay out of each other's way: nearly perpendicular (89 to 91 degrees) is enough, and you can fit far more of those than there are dimensions.

This idea - many features squeezed into fewer dimensions - is called **superposition**. More in the interpretability chapter.

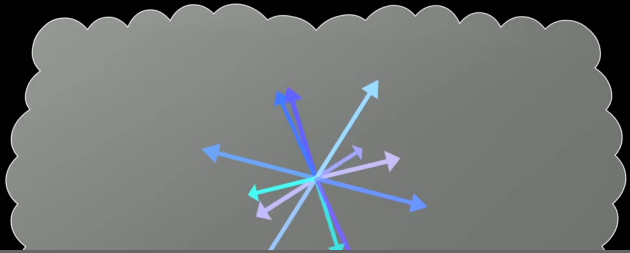


Choose multiple vectors,
each pair between 89° and 91° apart

Johnson–Lindenstrauss Lemma

$\Longrightarrow$ Maximum # of vectors: $\approx \exp(\epsilon \cdot N)$

N-dimensional Space

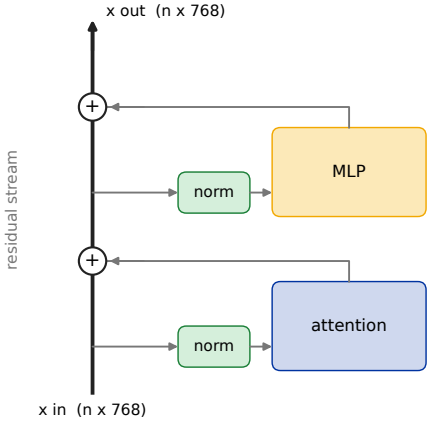


Johnson-Lindenstrauss: the number of nearly perpendicular directions grows exponentially with the dimension.

The block

What you actually stack.

The Transformer block



Attention: tokens exchange information. The *only* place in the block where tokens see each other.

MLP: each token alone; two thirds of the weights.

Norm before each, and the result **added back**: LayerNorm from [44], the residual connection from [50].

Every piece but attention is something you already met. GPT-2's order: norm, sublayer, add.

The residual stream is the real object

Read the block as a **stream** of token vectors running straight through the network, and each sublayer *writing an update into it*:

$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{Attention}\big(\mathrm{LN}(\mathbf{x})\big) \qquad \text{then} \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathrm{MLP}\big(\mathrm{LN}(\mathbf{x})\big) \qquad (\mathrm{LN} = \text{LayerNorm})$$

The stream itself is never normalised: each sublayer reads a normalised copy and adds a **correction**. Gradients reach the early layers along the stream - the ResNet argument from [50] - and a token's vector gathers meaning layer by layer.

GPT-2 moved the norm "to the input of each sub-block" (Radford et al., 2019). The 2017 paper normalised after the add: $\mathbf{x} \leftarrow \mathrm{LN}(\mathbf{x} + f(\mathbf{x}))$.

By hand: LayerNorm on creature

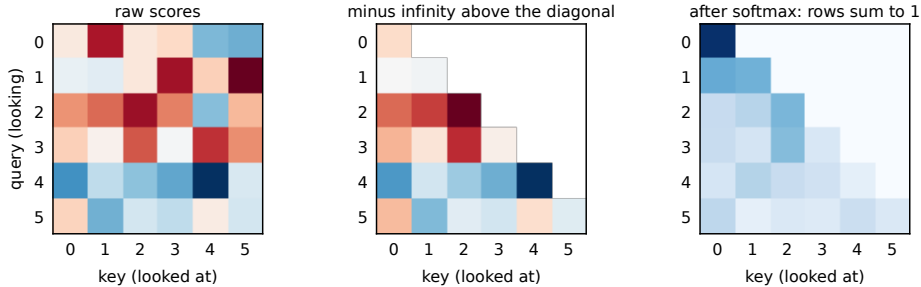
LayerNorm ([44]) rescales one token's vector so its numbers have mean 0 and spread 1. Creature after LLM-3, $(0,\ 0.47,\ 0.47,\ 2.13)$:

mean	$(0 + 0.47 + 0.47 + 2.13)/4 = 0.77$
spread	$\sqrt{\frac{1}{4}\sum(x_i - 0.77)^2} = 0.81$
normalised	$(x - 0.77)/0.81 = (-0.95,\ -0.37,\ -0.37,\ 1.68)$
then	times a learned scale, plus a learned shift (1 and 0 here)

Each token is normalised on its own numbers - not across the batch, as BatchNorm does ([44]). So it works the same for one sentence or a thousand.

Masking: a decoder must not read the future

A model trained to predict the next token must not be allowed to look at it. The fix is one line: add $-\infty$ above the diagonal *before* the softmax.



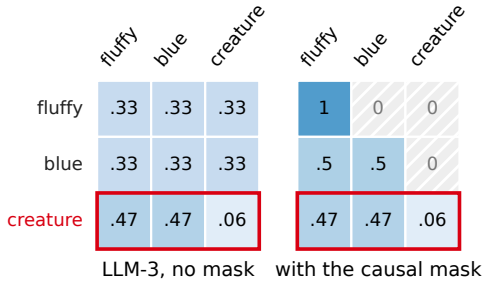
$e^{-\infty} = 0$, so those weights become **exactly** zero, and each row still sums to one.

By hand: the mask on creature's grid

LLM-3's grid (queries read "noun", keys read "adjective"), now with the mask:

- fluffy sees only itself: $(1, 0, 0)$;
- blue sees fluffy and itself, scores $0, 0$: $(0.5, 0.5, 0)$;
- creature's row does not change: $(0.47, 0.47, 0.06)$.

Its adjectives came first, so the mask took nothing from it.



In English adjectives come before their noun, so a left-to-right model can still let the noun gather them. What the mask removes is the other direction.

Why that trick matters more than it looks

Without masking you would train on one prediction at a time: feed the first t tokens, predict token $t+1$, repeat.

With masking, one forward pass over n tokens gives n **training signals at once**: every prefix is predicted at the same time, and none of them can cheat.

A large part of why pretraining on internet-scale text is affordable at all. The causal mask is not a safety measure; it is a **throughput** decision that happens to enforce correctness. (LLM-5 and LLM-7 come back to it.)

Modern block recipe

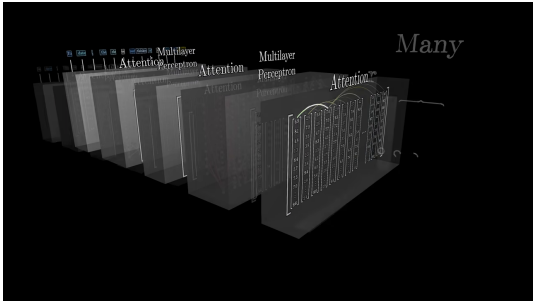
The block has been tuned since 2017. Three changes you will meet in current models:

2017	Now	Why
norm after the add	norm first (GPT-2 already)	trains more stably; can even skip warm-up
LayerNorm	**RMSNorm**	cheaper: no mean, only the spread
ReLU in the MLP	**SwiGLU**	a gated MLP; better quality in Shazeer's tests

Xiong et al., ICML 2020 (norm first); Zhang & Sennrich, NeurIPS 2019 (RMSNorm); Shazeer, 2020 (GLU variants).

None of these changes the story: attention, an MLP, norms and the residual stream. LLM-14 opens a 2026 model card and finds the same block.

Now stack it



3Blue1Brown

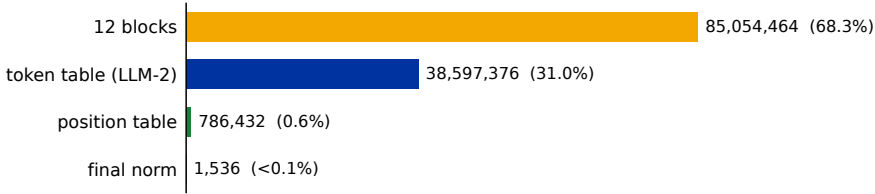
A block takes $n \times 768$ numbers in and gives $n \times 768$ out - the same shape, which is why blocks can be stacked. GPT-2 small stacks **12**.

Each block alternates **tokens talking** (attention) with **each token thinking alone** (the MLP).

A token's vector starts as "the word *creature*" and, layer by layer, gathers what the context implies: fluffy, blue, cute, and whatever comes next.

Same block, 12 times. That is the whole architecture.

Where GPT-2's 124M live



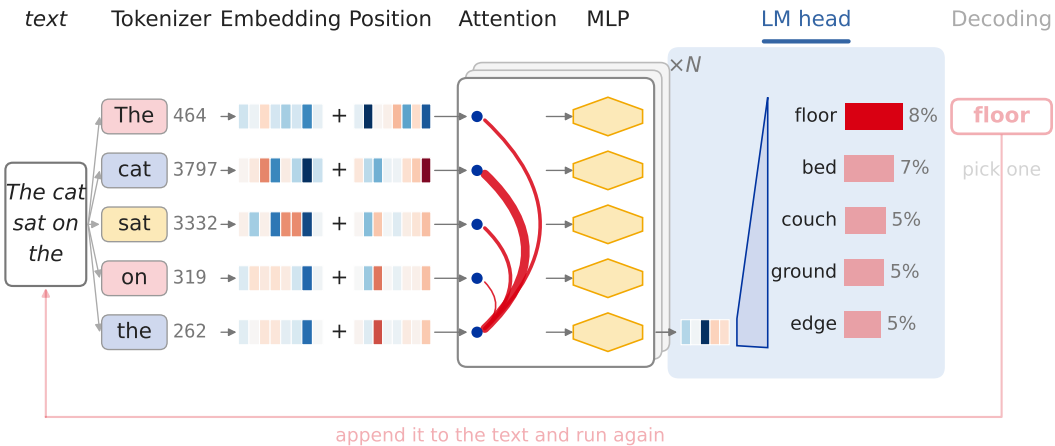
$12 \times 7{,}087{,}872 = 85{,}054{,}464$ in the blocks; the token table (LLM-2's 31%); 1,024 learned positions; one final norm. Total **124,439,808**, read from the weight file.

Two thirds of every block is MLP, and the blocks are two thirds of the model. Attention, the star of LLM-3, is under a quarter of GPT-2 (22.8%).

Recap

- **Many heads**: h questions at once, each in a smaller space (GPT-2: $12 \times 64 = 768$), mixed by W_O. Real heads: one looks one token back, one sends creature to fluffy, many park on the first token.
- **Still blind to order**: one head or twelve, so positions come in before the first layer (LLM-2).
- **The MLP**: rows ask, ReLU gates, columns add. A neuron can fire on "fluffy AND creature", and facts like Messi's sport seem to live here. Two thirds of every block.
- **The block**: norm, attention, add; norm, MLP, add. The stream is never normalised; every sublayer writes a correction.
- **The mask**: no reading the future, and n training signals per pass.

Next on the map



Next (LLM-5): what comes out at the far end, and three ways to wire the same block.